

REACTJS

INTRODUCTION

Introduction

- Name / Nickname
- Current Job / Position
- Tech Stack
- Expectations
- Choose an animal that best represents you and explain why



JEM LAGUDA

- Senior Software Engineer
- 6 years of experience in MERN Stack
- 4 years of experience in Teaching

Attendance

Hello!
Mark your attendance!

*You are on
your way to
a brighter
tomorrow!*



 **Metrobank**
You're in good hands

 **TRAINOSYS**
TRAINING THE FUTURE TODAY

Pre-assessment

Program

Day 01- React Fundamentals

- Introduction to React
- ReactJS features and benefits
- Create our first React project
- Understanding project structure
- JSX, Virtual DOM
- Class based and functional based components
- State, Props, Event Handling
- One way and two way binding
- React Hooks and Life Cycle Methods

Day 02 - Routing & Styling

- React Router
- Setting up Routes
- Handling route parameters
- Implementing nested routes
- Overview of styling in React
- Inline styling
- Styled-components
- Tailwind

Introduction to React

Example Projects

- <https://ecommerce-swift-mart.netlify.app/>
- <https://youichiuehara14.github.io/p3-react-app/#/>
- <https://wedlyapp.onrender.com/>

Intro to React

Objectives

- Learn about the React Library
- Learn what React does and why you would use it
- Learn how to get started using React

What is React?

- React is a *library* for building user interfaces. It lets you write HTML elements inside JavaScript files (and more!).

What is a *library*?

- In software, a 'library' is a published, self-contained set of code that you can import to help write your programs. Usually a library focuses on doing one thing well (React does UI).
- From here on out you'll use libraries often. With JavaScript, we'll use the **npm** command to find and install libraries.

React Example Code

```
import React from "react";

const HelloWorld = () => {
  return (
    <div>
      <h1>Hello world!</h1>
      <p>And good morning!</p>
    </div>
  );
}
```

React Example Code

```
import React from "react";
```



Imports **React** Library

```
const HelloWorld = () => {
  return (
    <div>
      <h1>Hello world!</h1>
      <p>And good morning!</p>
    </div>
  );
}
```

React Example Code

```
import React from "react";
```

Imports **React** Library

```
const HelloWorld = () => {
  return (
    <div>
      <h1>Hello world!</h1>
      <p>And good morning!</p>
    </div>
  );
}
```

Component name

React Example Code

```
import React from "react";
```

Imports **React** Library

```
const HelloWorld = () => {
```

```
  return (
```

```
    <div>
```

```
      <h1>Hello world!</h1>
```

```
      <p>And good morning!</p>
```

```
    </div>
```

```
  );
```

```
}
```

Component name

Defining our **JSX**

What is JSX?

- JSX (JavaScript XML) allows writing HTML-like syntax in JavaScript
- Simplifies UI development by combining HTML and JavaScript
- Enables dynamic content rendering and component composition
- Supports event handling and logic within the same file
- Enhances code readability and efficiency

That's not valid JavaScript!

- If you try pasting React code into a JavaScript file and opening it in your web browser, you'll probably get errors!
- You'll need to *build* your React apps so they run in the web browser
 - building your React apps will convert the JSX into something the browser knows how to handle (but the JSX is a lot easier to write)

How does it go from

JSX

JavaScript

```
return (<div>Hi</div>);
```



```
const div=document.createElement("div");  
div.textContent = "Hi";  
return div;
```

How does it go from

JSX

JavaScript

```
return (<div>Hi</div>);
```



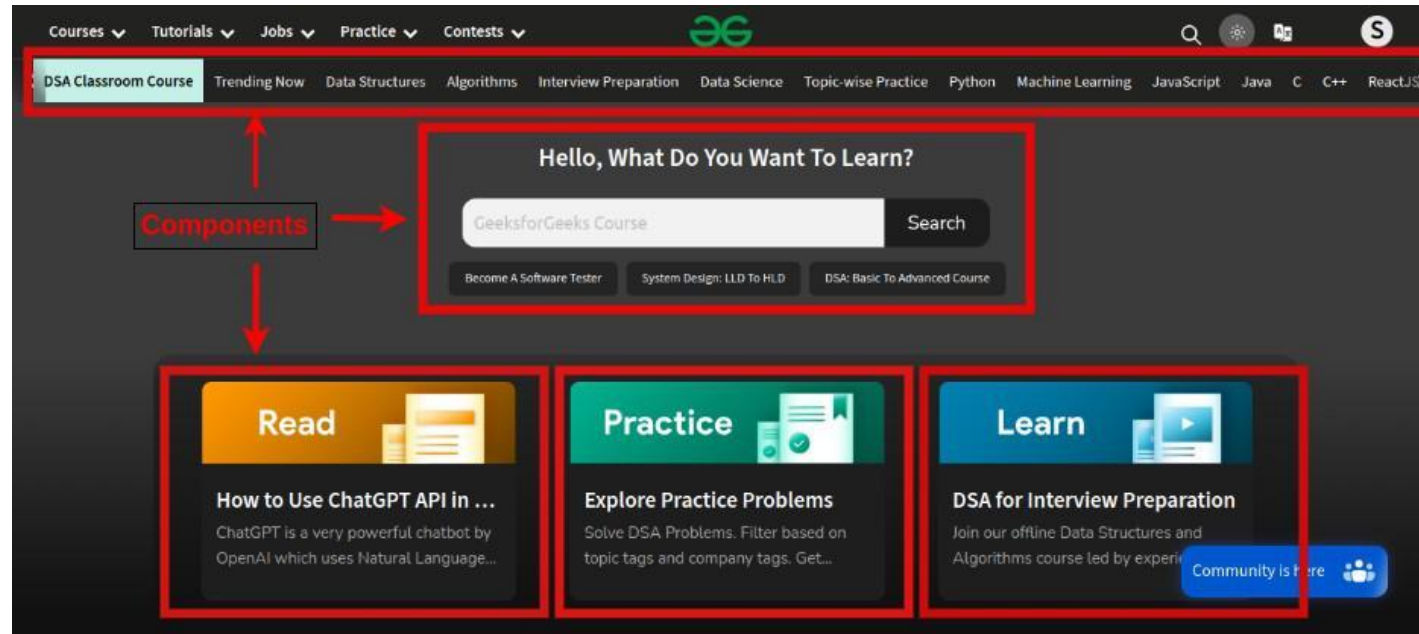
```
const div=document.createElement("div");  
div.textContent = "Hi";  
return div;
```

What is Babel?

- React uses JSX, a syntax extension that allows you to write HTML-like code directly in your JavaScript. Browsers don't natively understand JSX, so Babel compiles JSX into standard JavaScript that browsers can run
- Code runs seamlessly across different browsers and environments

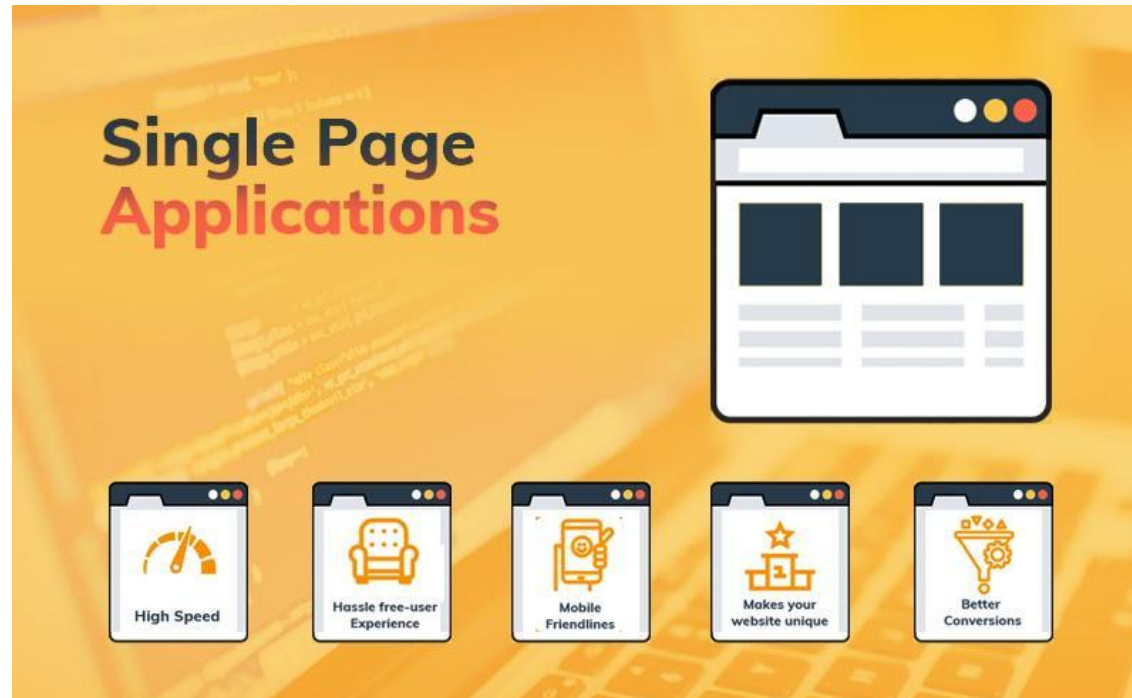
Why use React? Why is it so popular?

Component based approach which helps in building reusable UI components (each UI works in isolation)

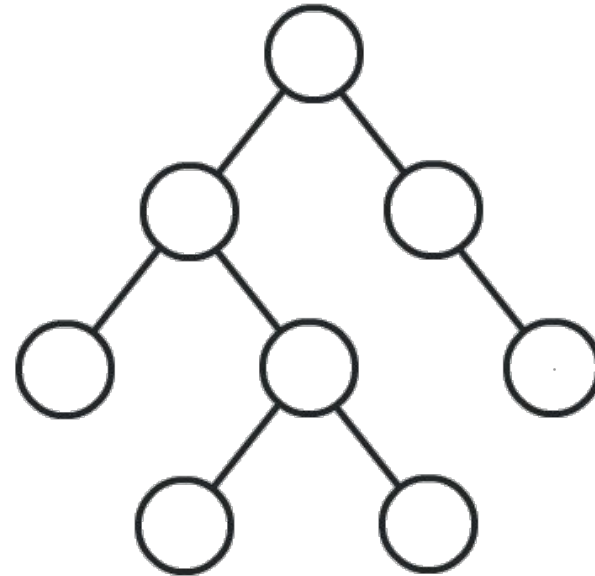


Credits to Geeks for geeks

SPA - Single Page Applications (React Router Library)

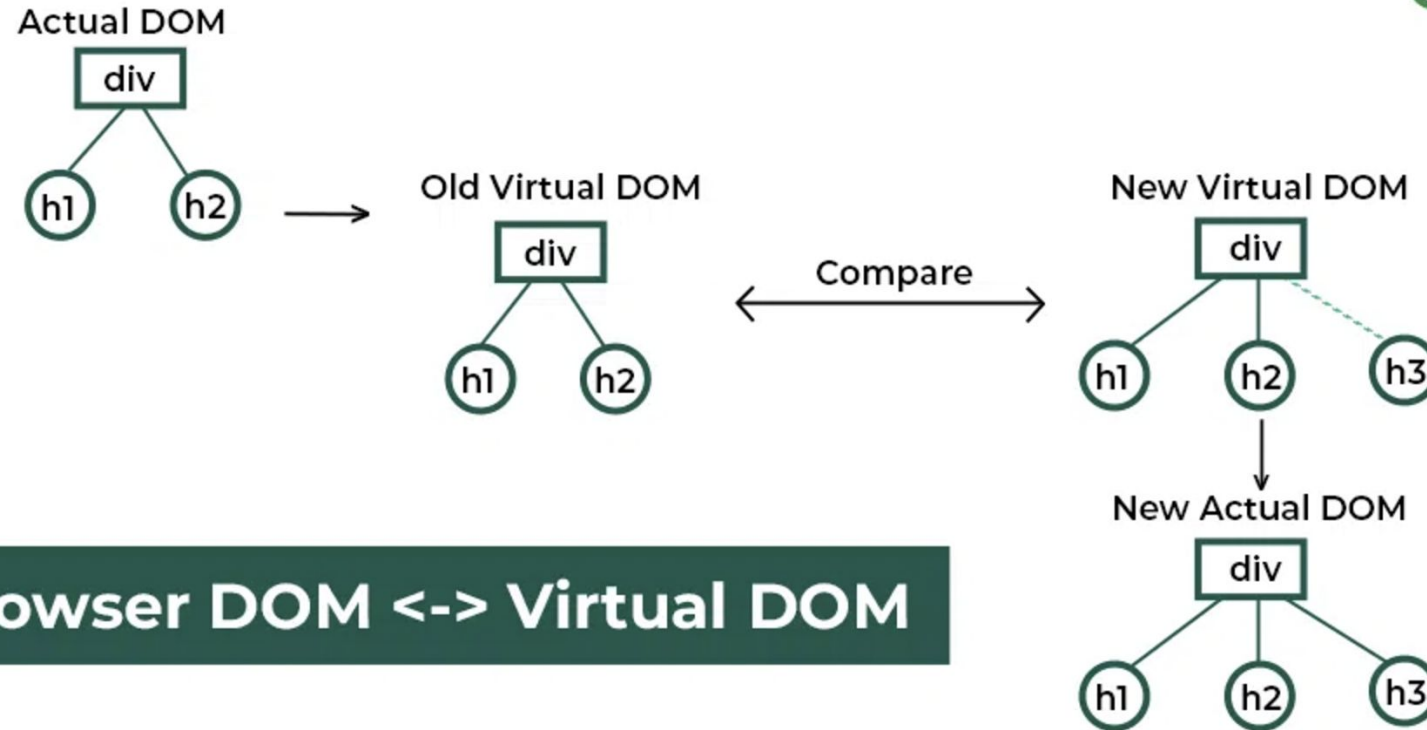


Unidirectional Flow



-  State change initiated
-  State change

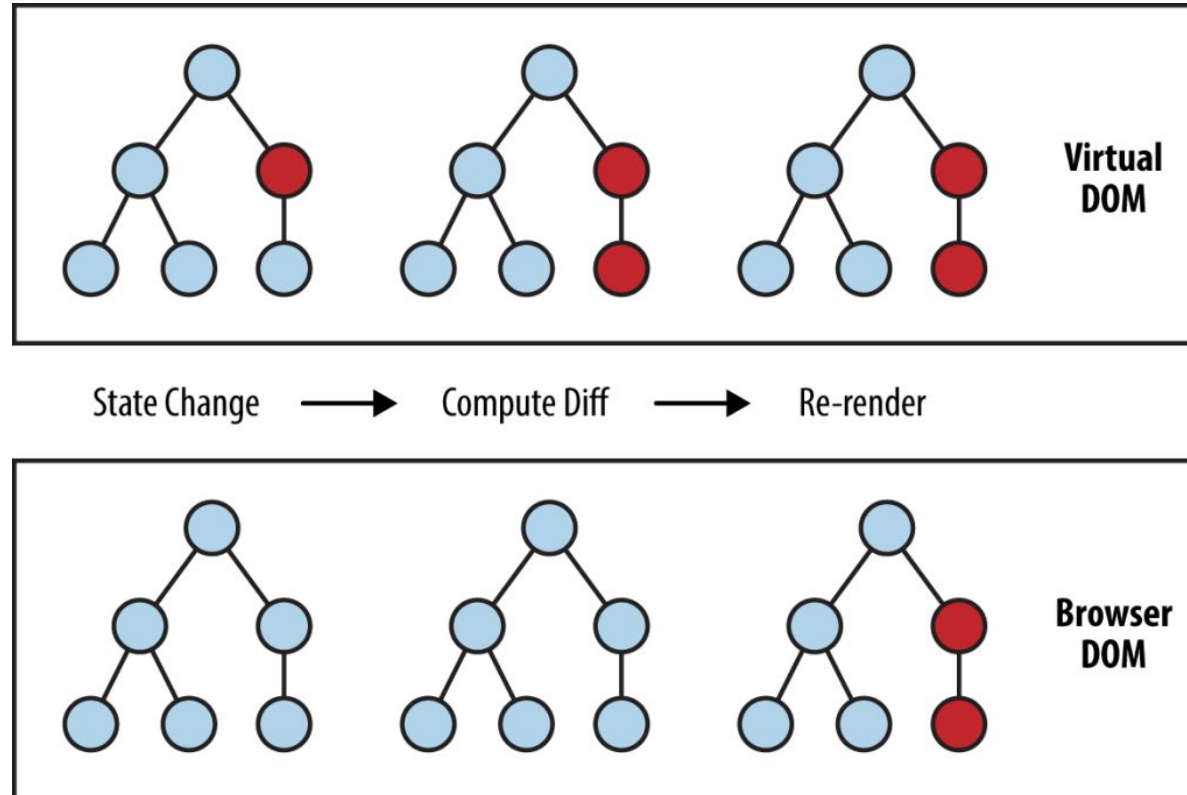
Virtual DOM



Browser DOM <-> Virtual DOM

Credits to Geeks for geeks
Image source: <https://media.geeksforgeeks.org/wp-content/uploads/20230725135348/Browser-DOM-Virtual-DOM-React.webp>

Virtual DOM



Code Along

Checklist of our tools

- NodeJS - <https://nodejs.org/en/download/prebuilt-installer>
- IDE - Like VS code
- Browser
- Postman or we could use thunder client in VS code
- Git - <https://git-scm.com/downloads>
- Github
 - Create github account
 - Make a public repository and push codes

React project via CRA (create react app)

```
npx create-react-app <app-name>
```

React project via VITE

```
npm create vite@latest
```

Folder Structure

my-react-app/	
— public/	# Static assets (optional, may not be generated initially)
— src/	# Application source code
— App.css	# Styling for App component
— App.jsx	# Main App component
— index.css	# Global styles
— main.jsx	# Entry point for the React app
— .gitignore	# Git ignore file
— index.html	# Main HTML template
— package.json	# Project dependencies and scripts
— README.md	# Project documentation
— vite.config.js	# Vite configuration file
— node_modules/	# Installed dependencies (generated after `npm install`)

ADD REACT TO YOUR JS PAGE

React is a library that turns React Components into HTML
 The following line actually integrates React with your page:

```
ReactDOM.render(<App />, document.getElementById('root'));
```

ReactDOM is a critical related library that works with React to add the App to the tree. It is imported this way:

```
import ReactDOM from 'react-dom';
```

ONLINE PLAYGROUNDS

- If you just want to try something out, there are some websites where you can play around with React easily without setting up a project:
- jsfiddle.net is great for quick tests, choose the 'React + JSX' boilerplate
- codesandbox.io is more like a full IDE in the browser
- There are plenty of other options if you search the web

What is a component?

What is a component?

In ReactJS, a component is a reusable, independent piece of UI that defines how a part of the user interface (UI) looks and behaves.

Key Characteristics

- Reusability: Build once, use multiple times.
- Isolation: Manages its own logic and rendering, independent of other components.
- Composable: Combine multiple components to create complex UIs.

Class based components vs Functional based components

Class based vs Functional based

Class Component

```
import React from "react";

class HelloWorld extends React.Component {
  render() {
    return (
      <div>
        <h1>Hello world!</h1>
        <p>And good morning!</p>
      </div>
    );
  }
}
```

Functional Component

```
const HelloWorld = () => {
  return (
    <div>
      <h1>Hello world!</h1>
      <p>And good morning!</p>
    </div>
  );
}
```

Class based Component

```
import React, { Component } from "react";

class ClassComponent extends Component {
  constructor(props) {
    super(props);
    this.state = {
      count: 0,
    };
  }

  incrementCount = () => {
    this.setState((prevState) => ({ count: prevState.count + 1 }));
  };

  render() {
    return (
      <div>
        <h1>Class Component</h1>
        <p>Count: {this.state.count}</p>
        <button onClick={this.incrementCount}>Increment</button>
      </div>
    );
  }
}

export default ClassComponent;
```

- **Syntax:** Use ES6 class syntax and extend `React.Component`.
- **Structure:** Require a `render()` method to return JSX.
- **State Management:** Manage state using `this.state` and `setState()`.
- **Lifecycle Methods:** Use built-in methods like `componentDidMount`, `componentDidUpdate`, `componentWillUnmount`, etc.
- **this Keyword:** Requires the use of `this` to access props and state, which can be confusing.
- **Code Verbosity:** More boilerplate code (e.g., constructors, method bindings).
- **Readability:** Generally more verbose and harder to read for new developers.
- **Performance:** Similar to functional components, but without built-in hooks for optimization.
- **Legacy:** Still in use but less favored for new projects.

Functional based Component

```
import React, { useState } from "react";

const FunctionalComponent = () => {
  const [count, setCount] = useState(0);

  const incrementCount = () => {
    setCount(count + 1);
  };

  return (
    <div>
      <h1>Functional Component</h1>
      <p>Count: {count}</p>
      <button onClick={incrementCount}>Increment</button>
    </div>
  );
};

export default FunctionalComponent;
```

- **Syntax:** Simple JavaScript functions that return JSX.
- **Structure:** No render() method needed; JSX is returned directly.
- **State Management:** Use hooks like useState for state management.
- **Lifecycle Methods:** Simulate lifecycle behavior using hooks such as useEffect.
- **this Keyword:** No need for this, making code simpler and easier to read.
- **Code Conciseness:** Less boilerplate, leading to cleaner and more maintainable code.
- **Readability:** Easier to read and write, especially with hooks.
- **Performance:** Optimized using hooks like useCallback and useMemo for better performance.

JSX

What is JSX?

- JSX, which stands for JavaScript XML, is an extension of JavaScript that allows developers to **write HTML-like code within their JavaScript files**. It enables the creation and composition of HTML elements and components using a syntax similar to HTML, making it easier to build user interfaces.
- By using JSX, developers can write code that looks like a combination of HTML and JavaScript, with the ability **to include variables, expressions, and logic** directly within the markup. This allows for dynamic content rendering, component composition, and event handling within the same file, enhancing the efficiency and readability of the code.

React JSX

React JSX lets you write DOM elements in JS using shorthand:

```
return (<button><TrainosysIcon /> Learn more!</button>);
```

REACT JSX - INTERPOLATION

- React also lets you embed JavaScript in your JSX using curly braces:

```
const userName = "Mary";
return (<h3>My name is {userName}!</h3>);
```

- You can write any valid JS expressions inside the curlies:

```
return (<p>I am {10 + 10 + 7} years old.</p>)
```

REACT JSX - EXPRESSIONS ONLY

- Interpolation only supports expressions!
- An expression is something you could return or assign

My name is Mary

- Valid:

```
return (<p>Hi {getUserName()}!</p>)
```

I am 27 years old.

- Not allowed!

```
return (<p>Hi {if (x > 3) { return 5;} }!</p>)
```

- No `if`, `for`, `foreach`, `switch`, `return`, etc

REACT - LISTS

- React will render arrays of elements in curly braces
- It's very common to use **.map** to turn items into DOM Elements

```
const items = ["rice", "chicken", "cookie"];
return (
  <div>
    <h2>Menu:</h2>
    <ol>
      {items.map(item => (<li>{item}</li>))}
    </ol>
  </div>
);
```

Menu:

1. rice
2. chicken
3. cookie

ACTIVITY 2- React JSX

branch name: activity-2

Activity 2 Part 1 - React JSX

Preparation:

Generate a react project using, make sure to change your folder name

```
npm create vite@latest react-form
```

Instruction

Update your App.jsx and returns a form with the following fields

- First name - A text input element
- Last name - A text input element
- Email - A text input element
- Password - A password input element
- Submit - A submit button

Challenge yourself

- Use an array of objects to simplify the creation of input fields

Activity 2 Part 2 - React JSX

Preparation:

Generate a react project using, make sure to change your folder name

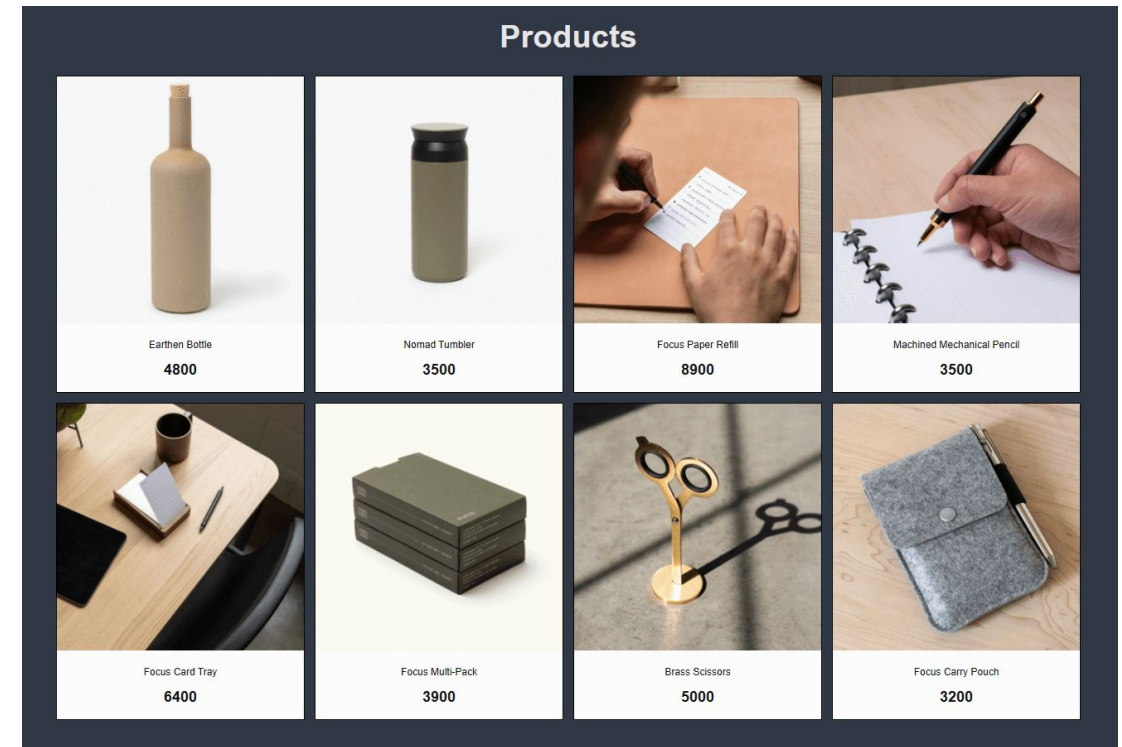
```
npm create vite@latest react-products
```

Instruction

- Update your App.jsx and returns list of products
- Each product should include an image, a title, and price
- Use the provided list of products

Challenge yourself

- Add a button to each list item that allows the user to add the product to their shopping cart.
- Allow the user to sort the products by price or name.
- Add CSS styles to the items, such as border, background, color, or font size.



REACT DATA

React Data

Learn how to handle React data

- Props
- State
- Events
- Two-way binding
- Conditionals

REACT - PROPS

A React component can have *props* (short for properties) just like an HTML element has *attributes*

Access them on `props`

```
const ScoreWidget = props => {
  return (<div>Score: {props.points}</div>)
}
```

```
<ScoreWidget points={12} />
```



REACT - STATE

A React component can have state. State can change.

```
import { useState } from 'react';
const ScoreWidget = () => {

  const [points, setPoints] = useState(12);

  return (<div>Score: {points}</div>)

}
```

Initialize state at the top and be sure to import useState

REACT - STATE

To change state in a component, always call `setPoints()`

```
const ScoreWidget = () => {
  const [points, setPoints] = useState(0);

  const addScore = () => {
    setPoints(points + 1);
  }
}
```

You ~~can't~~ shouldn't edit state directly.

REACT - STATE

- In React you should never modify any data on `state`
- Instead, make a **new copy** and set that as new state

BAD	SAFE
<pre>const [names, setNames] = useState(["Mark", "Luke"]); ... names.push("Cindy");</pre>	<pre>const [names, setNames] = useState(["Mark", "Luke"]); ... const newNames = names.concat(["Cindy"]); setNames(newNames);</pre>

REACT - EVENTS

- A react component can specify event handlers

```
const handleClick = () => {
  setClicks(clicks + 1);
}
```

```
return <button onClick={handleClick}></button>
```

- Just like DOM, use `onClick` for buttons, `onChange` for inputs, etc

DATA BINDING

- React: good for building forms
- Always a good reference: <https://reactjs.org/docs/forms.html>

EVENTS - DATA BINDING

```
const [firstName, setFirstName] = useState("a");
const [lastName, setLastName] = useState("b");

const firstNameChanged = (e) => {
    setFirstName(e.target.value);
}

render (
    <input type="text"
        value={firstName}
        onChange={(e) => firstNameChanged(e)} />
)
```

REACT JSX - CONDITIONALS

- React won't render `false`, `null` or `undefined` values in curlyes
- You can't use `if` but you can use *conditionals* and *ternary expressions*:

```
const hasName = false;
return (<p>Hi {hasName && "Kevin"}</p>);
```

```
const isAnonymous = true;
const userName = "Karen";
return (<p>by {isAnonymous ? "anonymous" : userName}</p>);
```

ACTIVITY 3

React JSX & Components

branch name: activity-3

Activity 3 Part 1 - React JSX & Components

Preparation:

Generate a react project using, make sure to change your folder name

```
npm create vite@latest react-form-2
```

Instruction

- Create a new component, you can simply name them Input. Follow the convention
- Add a prop for label where the name of the field
- Add a prop for type or the type of the input
- Add a prop for placeholder
- Render an input element with all the props
- Update your App.jsx and add your component

Challenge yourself

- Add additional props for input color, size and validation

First Name

Last Name

Email

Password

Activity 3 Part 2 - React JSX & Components

Preparation:

Generate a react project using, make sure to change your folder name

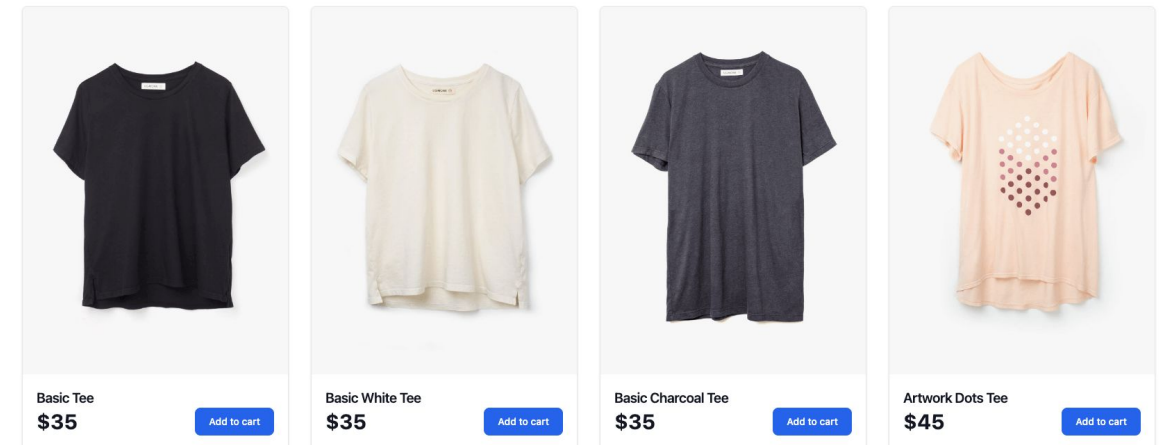
`npm create vite@latest react-products-2`

Instruction

- Create a new component, you can simply name them Card. Follow the convention
- Add a prop for title of the card
- Add a prop for price
- Add a prop for imageUrl
- Render a card element with all the props
- Update your App.jsx and add your component

Challenge yourself

- Use the map method on an array to create multiple elements
- Include extra information in the props, such as the currency or whether a product is available.
- Set default values for certain props, like using a default image if no imageUrl is provided.
- Allow additional customization with props for background color, link, and button text



Activity 3 Part 3 - React JSX & Components

Preparation:

Generate a react project using, make sure to change your folder name

`npm create vite@latest react-notifications`

✓ Successfully uploaded

Instruction

- Create a new component, you can simply name them Alert. Follow the convention
- Add a prop for title
- Add a prop for type (success, warning, error)
- Add a prop for message, but it is optional
- Render the element with all the props
- Update your App.jsx and add your component

⚠ Attention needed

Lorem ipsum dolor sit amet consectetur adipisicing elit. Aliquid pariatur, ipsum similique veniam quo totam eius aperiam dolorum.

Challenge yourself

- Add additional props for icon
- Add additional props to pass any content, you can use children props
- Add additional props to have an optional dismissable/close button

Activity 3 Part 4 - React JSX & Components

Preparation:

Generate a react project using, make sure to change your folder name

`npm create vite@latest react-ratings`

Instruction

- Create a new component, you can simply name them Rating. Follow the convention
- Add a prop for value
- Add a prop for maximumValue
- Render the element with all the props
- Update your App.jsx and add your component



Challenge yourself

- Add additional props for custom icon
- Add additional props for rating color
- Add additional props for size
- Change the value prop to accept decimal value and precise rating
- Add additional style like hover effect and animation

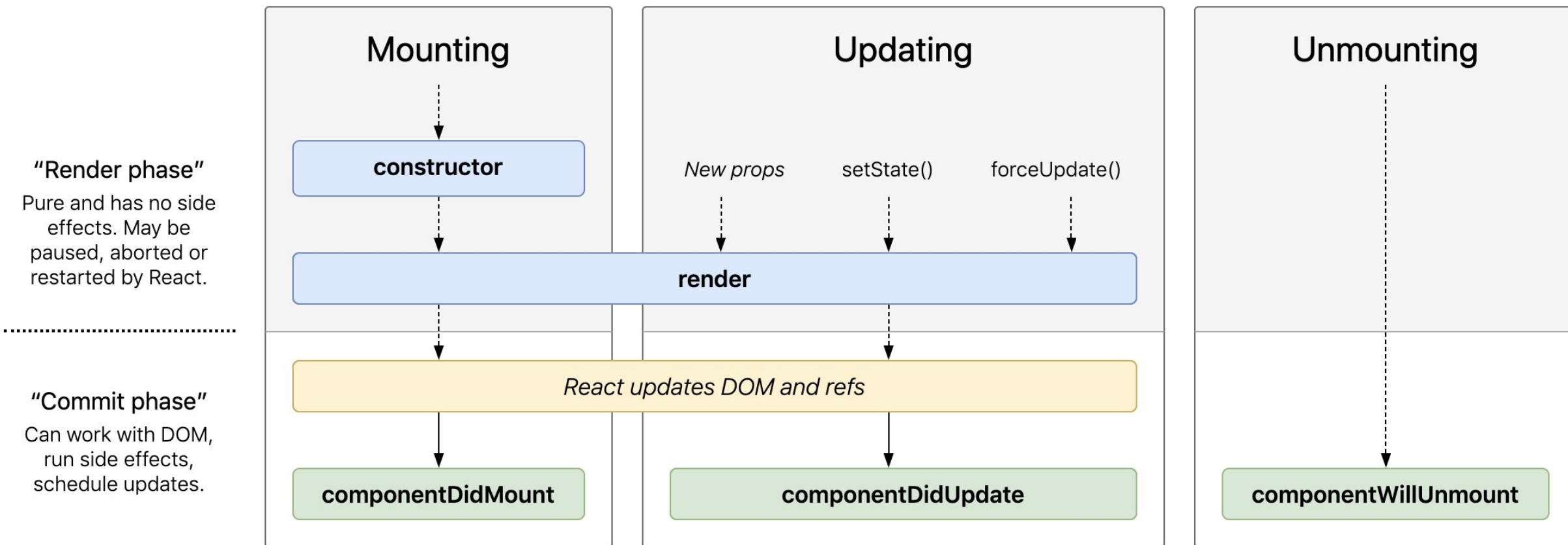
REACT HOOKS

Component Lifecycle

- Learn about React component lifecycle

LIFECYCLE EVENTS

- Lifecycle events are special React functions that get executed when things change:
 - `componentDidMount` - The component gets created
 - `componentWillUnmount` - The component gets removed
 - `componentDidUpdate` - The component data gets updated



Example: Timer

```
class Timer {
  componentDidMount() {
    this.timer = setTimeout(
      () => this.setState(this.state.time + 1)
    );
  }

  componentWillUnmount() {
    clearTimeout(this.timer);
    this.timer = 0;
  }
}
```

ADVANCE LIFECYCLE METHODS

These are less common but good to know about:

- `shouldComponentUpdate` – Decide if render needs to be called again

These define 'error boundaries' so a bug in one component won't crash your whole app:

- `componentDidCatch` – Called if a child component has an error that needs to be handled
- `getDerivedStateFromErrors` – Update state when errors happen

useState()

REACT HOOKS - useState

```
import { useState } from "react";

const [points, setPoints] = useState(0);
```

useState hook is used to define the state in a React functional component

States defined using this react hook is a tuple, the state name and function used to update the state variable

The value inside the useState function defines the initial value

useEffect()

REACT HOOKS - useEffect

```
import { useEffect } from "react";
```

```
useEffect(() => {
    alert('Hey, Nads here!');
});
```

```
useEffect(() => {
    alert('Hey, Jem here!');
}, []);
```

useEffect hook is used, within the React functional component, to define functions that run on specific component lifecycle

This useEffect function runs when a component is loaded and everytime a state value updates.

This useEffect function run only on initial load of the component.

REACT HOOKS - useEffect

```
import { useEffect } from "react";

const [points, setPoints] = useState(0);

useEffect(() => {
    alert('Hey, Kevin here!');
}, [points]);

useEffect(() => {
    alert('Hey, there!');
    return () => alert("Goodbye component")
});
```

This useEffect function runs only if state variable points changes.

This useEffect function only runs when the component is destroyed or removed from the UI

useContext()

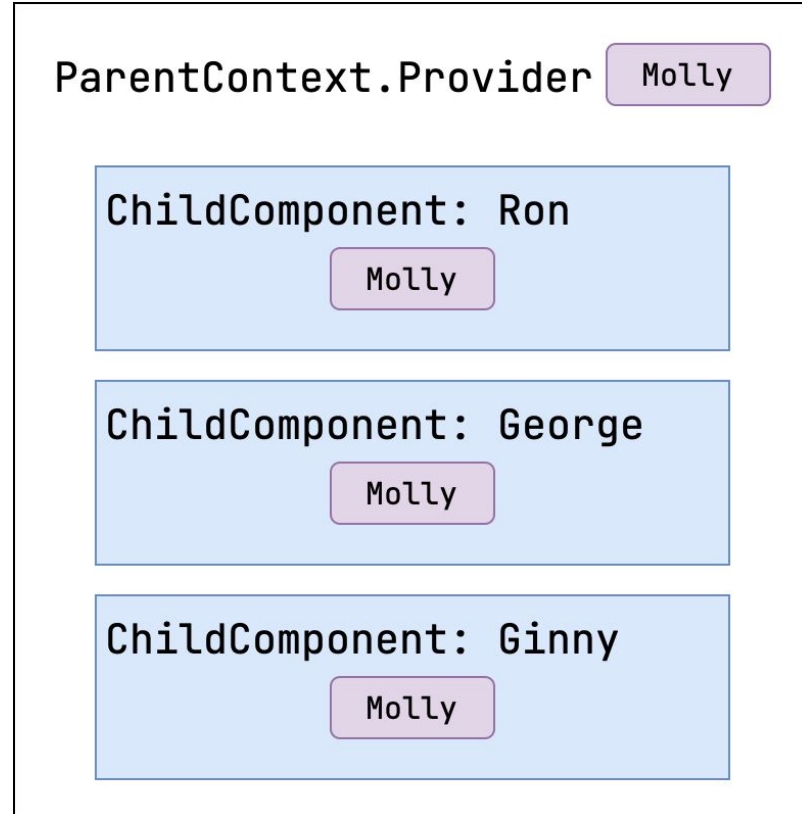
REACT HOOKS - useContext

Provides a way to pass data through its children

```
<ParentContext.Provider value="Molly">

  <ChildComponent name="Ron" />
  <ChildComponent name="George" />
  <ChildComponent name="Ginny" />
  ...
</ParentContext.Provider>
```

REACT HOOKS - useContext



```
<ParentContext.Provider  
value="Molly">
```

```
  <ChildComponent name="Ron" />  
  <ChildComponent name="George" />  
  <ChildComponent name="Ginny" />
```

```
  ...  
</ParentContext.Provider>
```

REACT HOOKS - useContext

- Create a context that later returns the current value of this context

```
import { createContext } from 'react';

export const ParentContext = createContext();
```

NOTE: You can pass default value as the first argument

REACT HOOKS - useContext

- Create a provider that will pass the value

```
export const ParentContext = createContext();
...
return (
  <ParentContext.Provider value="Molly">
    <ChildComponent />
    <ChildComponent />
    <ChildComponent />
    ...
  </ParentContext.Provider>
);
```


REACT HOOKS - useContext

- Get the current value of the context by using `useContext` hook

```
// ChildComponent.jsx
import { useContext } from 'react';
import { ParentContext } from './Parent';

const parentName = useContext(ParentContext);
```

REACT HOOKS - useContext

- Consume the value **WITHOUT** the hook
- Another way to consume the value of the context

```
// ChildComponent.jsx
import { ParentContext } from './Parent';
...
<ParentContext.Consumer>
  {value => (<p>{value}</p>)}
</ParentContext.Consumer>
;
```

Need to pass an expression, and **value** represents the current value of the context

REACT HOOKS - useContext

- Try to put a value using React data, e.g props or states, as it may cause unnecessary re-render every time

✗ BAD

```
<MyContext.Provider value="foo">
```

✓ GOOD

```
<MyContext.Provider value={props.name}>
```

```
const [name, setName] = useState();
<MyContext.Provider value={name}>
```

REACT HOOKS - useContext

- You can also pass any value and even a function

```
const [name, setName] = useState();
```

```
<MyContext.Provider value={{
  name,
  setName,
  customFunction
}}>
```

REACT HOOKS - useContext

- You can also pass any value and even a function

```
const [name, setName] = useState();
```

```
<MyContext.Provider value={{
  name,
  setName,
  customFunction
}}>
```

useReducer()

REACT HOOKS - useReducer

- useReducer was a new hook added on React version 16 release
- The hook was created to be used for more complex state handling
- Just like useState it used to control and maintain a certain state

useState vs useReducer

- useState is used for simple data structures
- useReducer is used for handling more complex data changes
 - What qualifies as “complex”?
 - Deeply nested objects
 - Array of complex objects
 - Multi-step changes to a specific key of an object
 - Keys which values depend on other keys
 - Database updates
 - HTTP calls to read and update data before using in the UI

REACT HOOKS - useReducer

`useReducer(reducerFunc, initialState)`

REACT HOOKS - useReducer

`useReducer(reducerFunc, initialState)`

`"elephant"`

`200`

`{
 name: "Karen",
 age: 17,
 sex: female
}`

`[1,2,3,4,5]`

REACT HOOKS - useReducer

`useReducer(reducerFunc, initialState)`



`(state, action) => { return state + action; }`

REACT HOOKS - useReducer

`useReducer(reducerFunc, initialState)`



```
(state, action) => {
  switch(action.type){
    case "add":
      return state + 1;
    case "edit":
      return state = 2;
  }
}
```

Custom Hooks

Custom Hooks

Definition:

- Custom Hooks are JavaScript functions in React that allow you to encapsulate and reuse stateful logic.
- They are created using built-in React hooks like `useState`, `useEffect`, `useContext`, etc.

Why Use Custom Hooks?

- Simplifies code and avoids duplication.
- Makes components more focused on UI rather than logic.
- Encourages reusability across components.

Example Name Convention:

Custom hooks always start with `use`, e.g., `useFetch`, `useForm`

Key Benefits of Custom Hooks

Reusability: Write logic once and reuse it in multiple components.

Separation of Concerns: Keep UI and logic separate.

Cleaner Code: Reduce clutter in functional components.

Testability: Logic in hooks is easier to test independently.

ACTIVITY 4 - React Hooks

branch name: activity-4

Activity 4 Part 1 - useState()

Preparation:

Generate a react project using, make sure to change your folder name

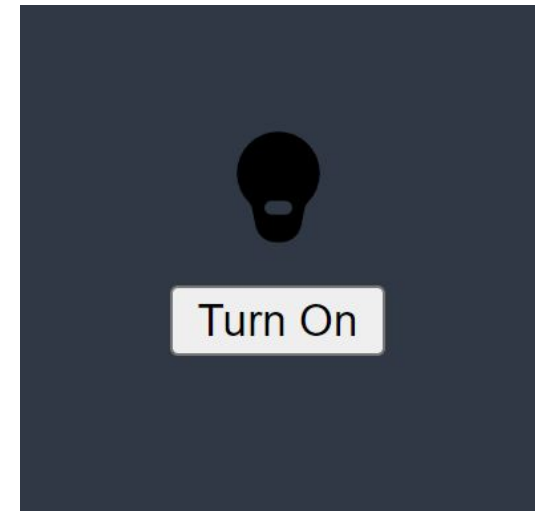
```
npm create vite@latest react-hooks
```

Instruction

- Create a new component, you can simply name them `LightSwitch`. Follow the convention
- Use `useState` hook to create a state variable, `isLightOn` and set an initial value to `false`.
- Render a button that will act as the light switch
- Display an image or text indicating whether the light is on or off, based on the state
- Write a function that toggles `isLightOn` from `true` to `false` and vice versa
 - a. This function should be called when the button is clicked

Challenge yourself

- Change the background color of the page or a specific div based on whether the light is on or off



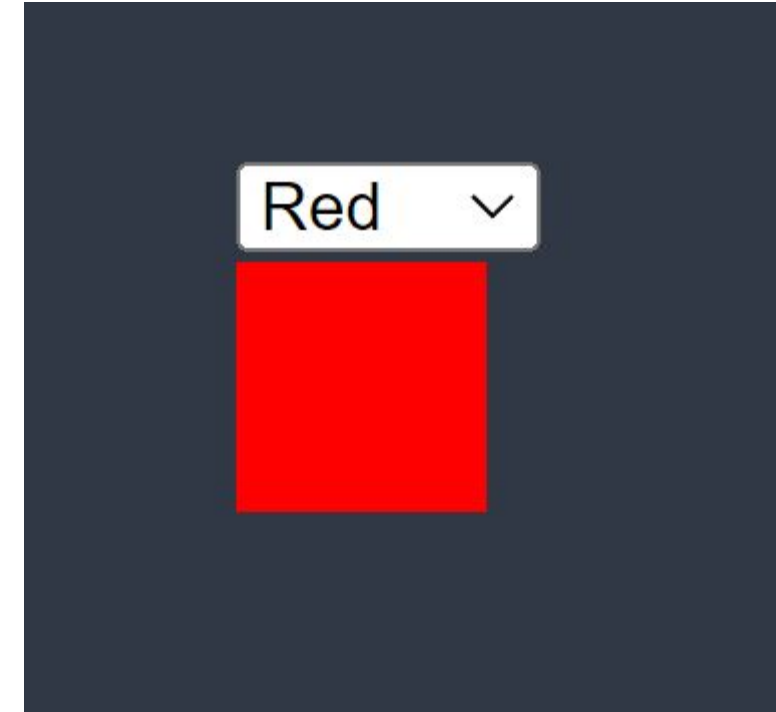
Activity 4 Part 2 - useState()

Instruction

- Create a component named ColorPicker. Follow convention
- Use and import the `useState` hook to create a state variable, initially set to a default color (e.g., 'red')
- Render a dropdown `<select>` element with a few color options (e.g., red, green, blue, etc.)
- Include a display box `<div>` that shows the selected color
- Write a function to update the state variable, based on the user's selection
- Attach this function to the `onChange` event of the dropdown `<select>`

Challenge yourself

- Add more styling



Activity 4 Part 3 - useEffect()

Instruction

- Create a component called MousePosition. Follow convention.
- Define a state variable, **position**, with an initial value of an **object** x and y properties, each properties have a value of zero (0)
- Use the **useEffect** hook to update the state variable, position, whenever the mouse moves.
- Render the state variable

Example Implementation

```
import MousePosition from './components/MousePosition';

function App() {

  return (

    <MousePosition />

  );

}

export default App;
```

Activity 4 Part 4 - useEffect()

Instruction

- Create a component called RandomColor. Follow convention.
- Define a state variable, color, with an initial value of red or your favorite color.
- Use the **useEffect** hook to update the state variable, color, for every 3 seconds
- Render an element with an inline background color style that matches the current value of the state variable, color.

Example Implementation

```
import RandomColor from './components/RandomColor';

function App() {
  return (
    <RandomColor />
  );
}

export default App;
```

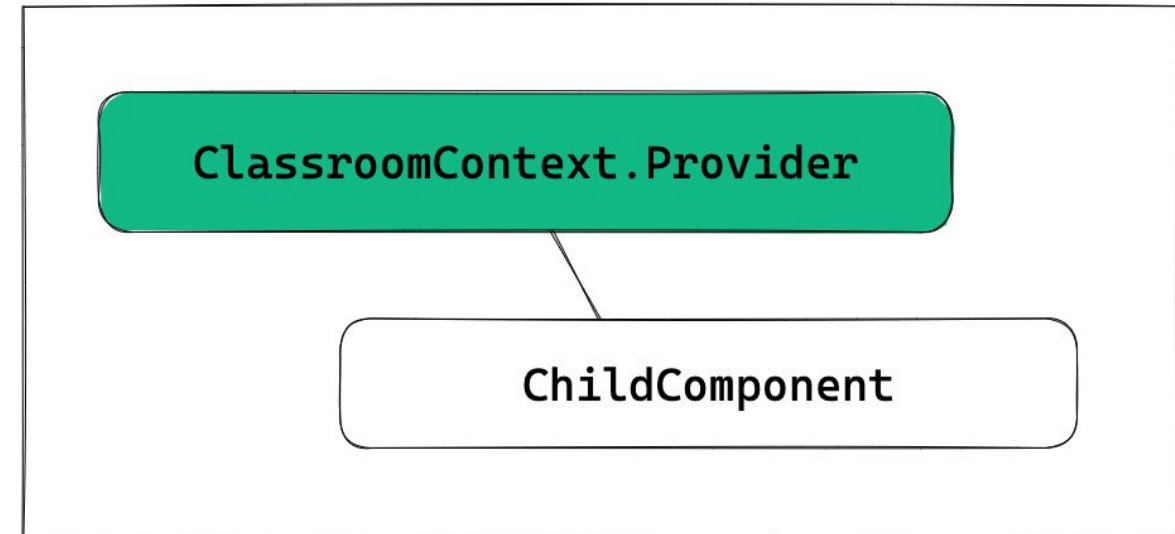
Activity 4 Part 5 - useContext()

Scenario

Imagine you're building a virtual classroom app. Each student has their own username, and the app has a theme setting the determines the background color. You want to make sure that the username and theme are accessible to different components without passing them around manually

Instruction

- Create a context object using createContext(). Think of this as a special container where we'll store and share the username and theme data.
- Define the shared data object, which includes the username and theme in an object format
- Wrap your main component or the components that need access to the shared data with the Context.Provider component. Pass the shared data object as the value prop
- Create a component called ChildComponent
- Use the useContext() hook to access the shared data
- Display the username and theme data in the child component



Activity 4 Part 6 - useReducer()

Instruction

- Create a component called Form
- Define the initial state. The initial state will be an object with name and email properties
- Define a reducer function. The reducer takes the current state and an action as parameters, and returns the new state based on the action. We'll have the following actions:
 - a. UPDATE_EMAIL
 - b. UPDATE_NAME
 - c. RESET_FORM
- **Remember NOT to modify the state**
- Use the useReducer hook. Pass the reducer function and the initial state as arguments to the hook. This hook will return an array with two (2) elements: the current state, and a dispatch function
- Render the Form

RESTFUL APIs

Objectives

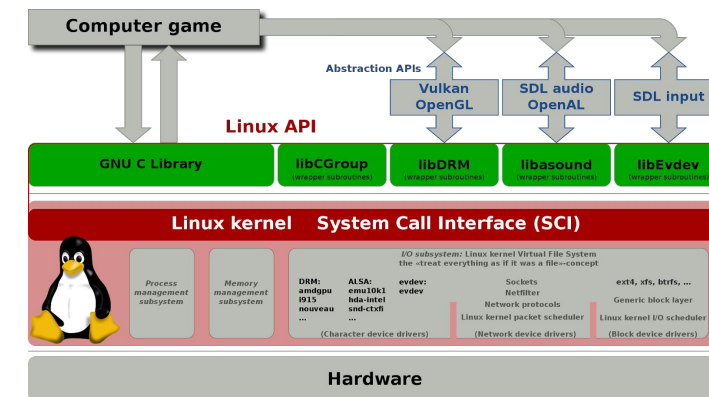
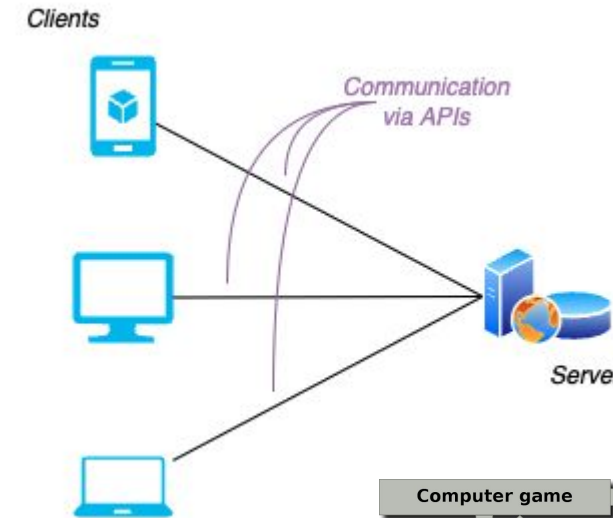
By the end of this module, we should have a good understanding of

-  what an API is
-  different ways to build an API
-  common http methods and CRUD
-  parts of a request and a URL
-  best practices in designing and building APIs

What is an API

API stands for Application Programming Interface.

- enable two software components to communicate with each other
- Application refers to any software with a distinct function
 - The application sending the request is called the client, and the application sending the response is called the server.
- Interface is a set of definitions and protocols



What are different ways to build APIs?

1. SOAP - Client and server exchange messages using XML

POST /InStock HTTP/1.1

Host: www.example.org

Content-Type: application/soap+xml; charset=utf-8

Content-Length: nnn

```
<?xml version="1.0"?>
```

```
<soap:Envelope
```

```
  xmlns:soap="http://www.w3.org/2003/05/soap-envelope/"
```

```
  soap:encodingStyle="http://www.w3.org/2003/05/soap-encoding">
```

```
  <soap:Body xmlns:m="http://www.example.org/stock">
```

```
    <m:GetStockPrice>
```

```
      <m:StockName>IBM</m:StockName>
```

```
    </m:GetStockPrice>
```

```
  </soap:Body>
```

```
</soap:Envelope>
```

What are different ways to build APIs?

2. RPC - allows a software to perform procedures in another system

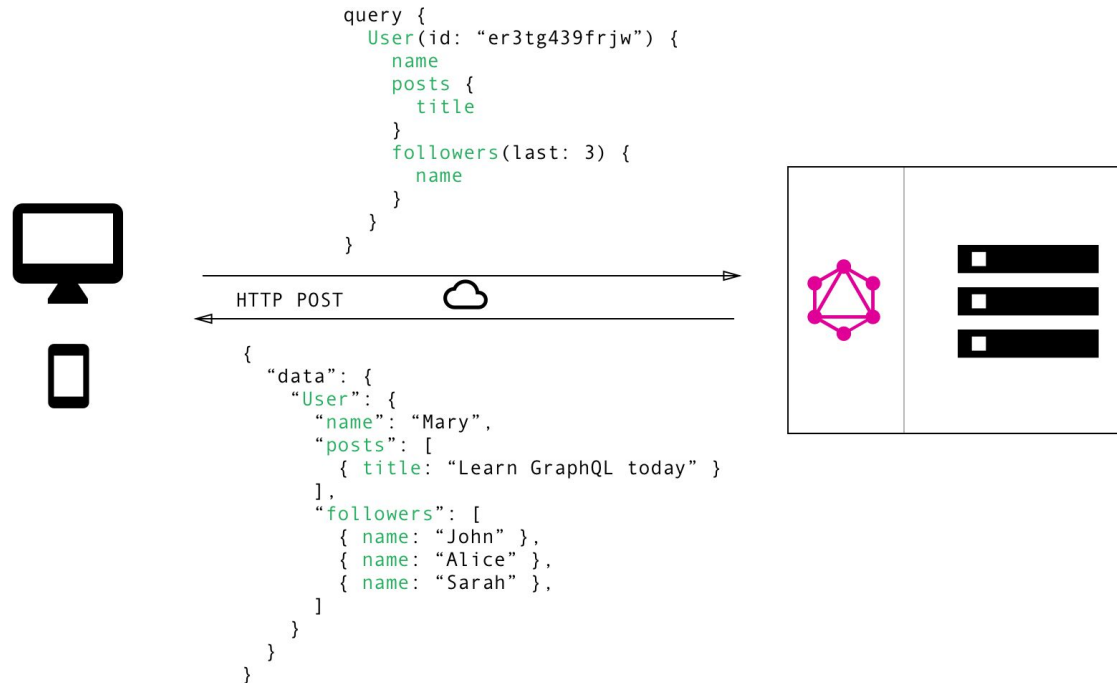
```

/* This program contains a procedure to add 2 numbers to
demonstrate
* some of the features of the new rpcgen. Note that add() takes 2
* arguments in this case.
*/
program ADDPROG {
    version ADDVER {
        int add ( int, int )
            = 1;
    } = 1;
} = 199;

```

What are different ways to build APIs?

3. GraphQL - query language that enables the client to describe what data to retrieve



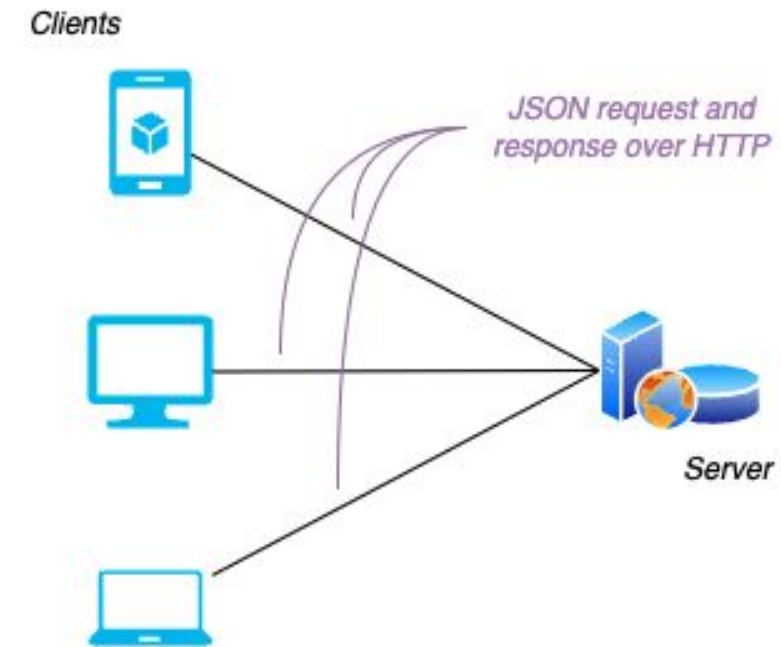
What are different ways to build APIs?

4. RESTful APIs

Representational state transfer (REST) is a software architectural style that defines a set of functions to be used for creating Web services (like *GET*, *POST*, etc)

Clients and servers exchange data using HTTP.

The main feature of REST API is *statelessness*. *Statelessness* means that servers do not save client data between requests.



HTTP

The **Hypertext Transfer Protocol (HTTP)** is an application layer protocol that is designed to enable communications between clients and servers.

It works as a request-response protocol between a client and server.

HTTP METHODS / VERBS

	PC						MOBILE					
	Chrome	Edge	Firefox	Internet Explorer	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Internet	WebView Android
CONNECT	✓ Yes	✓ 12	✓ Yes	✓ Yes	✓ Yes	✓ Yes	✓ Yes	✓ Yes	✓ Yes	✓ Yes	✓ Yes	✓ Yes
DELETE	✓ Yes	✓ 12	✓ Yes	✓ Yes	✓ Yes	✓ Yes	✓ Yes	✓ Yes	✓ Yes	✓ Yes	✓ Yes	✓ Yes
GET	✓ Yes	✓ 12	✓ Yes	✓ Yes	✓ Yes	✓ Yes	✓ Yes	✓ Yes	✓ Yes	✓ Yes	✓ Yes	✓ Yes
HEAD	✓ Yes	✓ 12	✓ Yes	✓ Yes	✓ Yes	✓ Yes	✓ Yes	✓ Yes	✓ Yes	✓ Yes	✓ Yes	✓ Yes
OPTIONS	✓ Yes	✓ 12	✓ Yes	✓ Yes	✓ Yes	✓ Yes	✓ Yes	✓ Yes	✓ Yes	✓ Yes	✓ Yes	✓ Yes
POST	✓ Yes	✓ 12	✓ Yes	✓ Yes	✓ Yes	✓ Yes	✓ Yes	✓ Yes	✓ Yes	✓ Yes	✓ Yes	✓ Yes
PUT	✓ Yes	✓ 12	✓ Yes	✓ Yes	✓ Yes	✓ Yes	✓ Yes	✓ Yes	✓ Yes	✓ Yes	✓ Yes	✓ Yes
TRACE	⚠ ?	⚠ ?	⚠ ?	⚠ ?	⚠ ?	⚠ ?	⚠ ?	⚠ ?	⚠ ?	⚠ ?	⚠ ?	⚠ ?

CRUD

	Operation	HTTP method	Description
C	Create	POST	used to send data to a server to create a resource.
R	Read	GET	used to request data from a specified resource.
U	Update	PUT / PATCH	used to send data to a server to update a resource.
D	Delete	DELETE	deletes the specified resource.

Important Parts of a request

1. A request line - method, path and http version
2. HTTP headers, or header fields
3. Message body

Parts of the URL

https://shopee.ph/api/v4/official_shop/get_shops?category_id=11021963&limit=23&offset=0

1. Scheme = https
2. Host = shopee.ph
3. Path = /api/v4/official_shop/get_shops
4. Query string = category_id=11021963&limit=23&offset=0

Path versioning

/api/**v1**/**X.Y.Z**

1. Version of API
2. Major version change
3. Minor version change
4. Hot fix version change

Sample: /api/v1/1.39.111

HTTP Status Codes

- Informational responses (100–199)
- Successful responses (200–299)
- Redirection messages (300–399)
- Client error responses (400–499)
- Server error responses (500–599)

Best practices in designing APIs

- Design the endpoint around resources - meaning use nouns/objects. And use plural nouns.
Good example: [/users](#) ; Bad examples: [/user](#) , [/eat](#)
- Version the endpoint
Good example: [/api/v1/users](#)
- Nest endpoint structure to show hierarchy
Good example: [/api/v1/users/:userId/comments](#)
- To manage request-responses with big data, use pagination, filtering or sorting as query parameters
Good example: [/api/v1/users/:userId/comments?limit=25](#)

Best practices in designing APIs

- Use JSON schema for the request and response
- Use proper HTTP methods
- Use proper HTTP status code for the response
- Do NOT send the whole error message as a response body
- Prepare API documentation

Tip: there are ways to do this programmatically eg. Swagger

FETCH API vs AXIOS

Fetch API:

A built-in JavaScript API for making HTTP requests. Introduced in ES6.

Axios:

A third-party library that simplifies HTTP requests in JavaScript.

FETCH API

```
const response = await fetch('https://api.example.com/data') ;

const data = await response.json() ;

console.log(data) ;
```


AXIOS

```
const response = await axios.get('https://api.example.com/data');  
console.log(response.data); // Axios automatically parses the JSON
```

FETCH API vs AXIOS - Overview

Feature	Axios	fetch()
API Type	Third-party library	Native to modern browsers
Response Format	JSON by default	Must parse response with .json()
Error Handling	Automatically handles HTTP errors (non-2xx statuses)	Requires manual error checking
Timeout Support	Built-in support via the timeout option	Manually handled
Promise Handling	Uses native promises or async/await	Uses native promises or async/await
Browser Support	Works in older browsers (example: IE 11) without polyfills	Not supported in IE 11 and other older browser versions, requires a polyfill for backward compatibility
Interceptors	Yes, supports request and response interceptors	No, requires manual implementation of similar behavior using wrapper functions

React Router

What is React Router?

Definition: React Router is a library for managing navigation and routing in React applications. It enables:

- Single Page Applications (SPA): Allows users to navigate between views without full-page reloads.
- Dynamic Routing: Routes change based on the application state.

Core Features:

- Declarative routing.
- Nested routes.
- Dynamic route parameters.
- Programmatic navigation.

Why Use React Router?

Declarative Syntax: Define routes using JSX.

SPA Navigation: Navigate seamlessly without reloading the page.

Dynamic Routing: Match routes dynamically to changing URLs.

Nested Routes: Create routes within routes for modular components.

React Router Dom

```
import { RouterProvider, createBrowserRouter } from 'react-router-dom';

import { Link } from 'react-router-dom';
```

Tailwind in React (Vite)

React Vite + Tailwind

<https://tailwindcss.com/docs/guides/vite>

ACTIVITY 5 - FETCH API, React Router and Tailwind

branch name: activity-5

REDUX

What is Redux?

Redux is a state management library for JavaScript Applications

In other words, redux is used to manage the data or state of complex JavaScript applications

PROs

- Centralize application's state
- Data-flow transparent
- Easily debug applications
- Preserve the page state

CONs

- Somehow complex
- Write a lot of boiler plate code

When should we use Redux?

- Complex User interfaces in terms of data like facebook, amazon, etc.
- Data flow is complex

When it's not recommended to use Redux?

- Small or medium size of applications
- Simple UI and static data

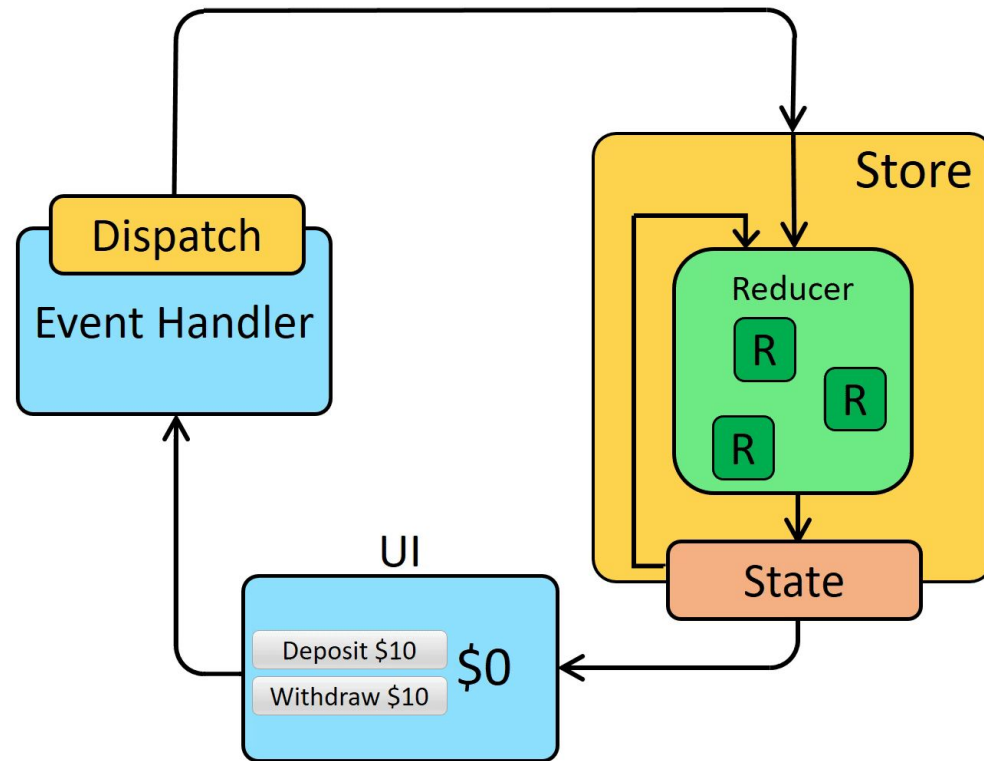
How Redux works?

Actions - What to do

Reducers - How to do

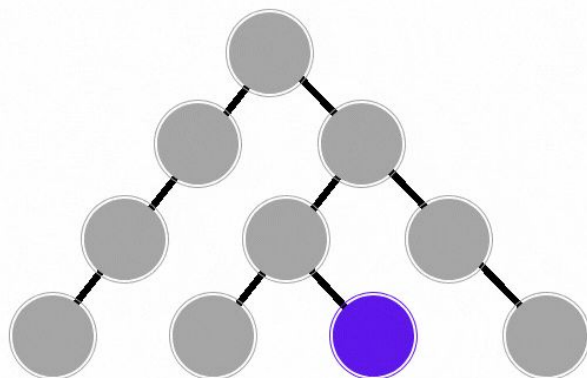
Store - Keep data in single place

How Redux works?



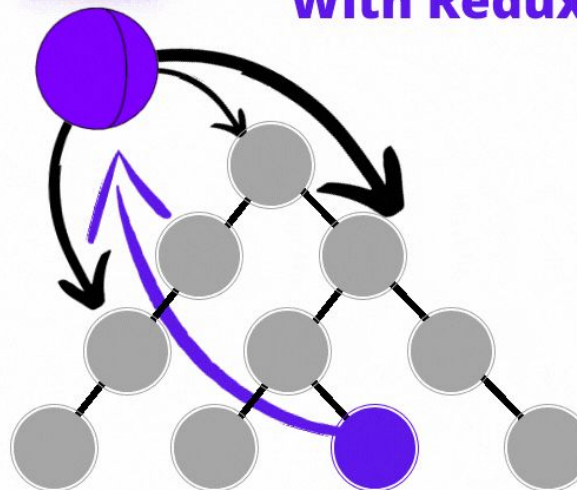
What is Redux?

Without Redux



STORE

With Redux



The Component initiating the change

Redux Toolkit

```
npm install @reduxjs/toolkit
```

```
npm i react-redux
```

Redux vs useReducer()

 REDUX TOOLKIT		 useReducer
Scope	Global (app-wide)	Local (component-level)
Boilerplate	Minimal	Low
Learning Curve	Moderate	Small to medium local state with complex transitions
Best For	Medium to large apps with shared global state	✗ No built-in support ✗ Manual with useEffect
Dev Tools	✓ Full Redux DevTools support	✗ Built-in with slice reducers
Async Logic (thunks)	✓ Built-in with createAsyncThunk	✗ Not applicable
Code Splitting	✓ Supports middleware	✗ Supports middleware (e.g., logging, async)

Debugging Redux

Install Redux dev Tools from Chrome Web Store

ACTIVITY 6 - Refactor to Redux

FOLDER NAME: activity-6-<LAST-NAME>

branch name: activity-6

REACTJS UNIT TESTING

React Testing with Vitest

A Modern Approach to Testing React Components

Why Test React Apps?

- Prevent regressions
- Ensure components behave correctly
- Catch bugs early
- Improve confidence in refactoring

What is Vitest?

- A blazing-fast unit testing framework for Vite
- Lightweight Jest alternative
- Built-in support for mocking, snapshots, TypeScript

Setting Up Vitest

```
npm install -D
vitest
@testing-library/react
@testing-library/user-event
@testing-library/jest-dom
jsdom
```

Add script in package.json

```
"scripts": {
  "test": "vitest"
}
```

Create a vitest.config.js file

```
import { defineConfig } from "vite";

export default defineConfig({
  test: {
    globals: true,
    environment: "jsdom",
    setupFiles: ["./src/setupTests.js"]
  },
});
```

Create a setupTests.js file

```
import @testing-library/jest-dom;
```

Testing Basics – Example Component

```
// Button.jsx
```

```
const Button({ label, onClick }) {  
  return <button onClick={onClick}>{label}</button>;  
}
```

```
export default Button
```

Writing Your First Test Case

```
// Button.test.jsx

import { render, screen, fireEvent } from '@testing-library/react'
import Button from './Button'

test('renders button with label', () => {
  render(<Button label="Click me" />)
  expect(screen.getByText('Click me')).toBeInTheDocument()
})
```


Simulating User Events

```
test('calls onClick when clicked', () => {
  const handleClick = vi.fn()
  render(<Button label="Click" onClick={handleClick} />)

  fireEvent.click(screen.getByText('Click'))
  expect(handleClick).toHaveBeenCalled()
})
```

Testing Form Inputs

```
render(<input type="text" placeholder="Name" />)
fireEvent.change(screen.getByPlaceholderText("Name"), {
  target: { value: "Mary" },
})
expect(screen.getByDisplayValue("Mary")).toBeInTheDocument()
```

Common Matchers

- `.toBeInTheDocument()`
- `.toHaveTextContent()`
- `.toHaveAttribute()`
- `.toBeDisabled()`
- `.toHaveBeenCalledTimes(n)`

Run your tests

```
npm run test
```

Summary

- **Vitest is fast, modern, and Jest-like**
- **Use Testing Library to test behavior, not implementation**
- **Keep tests readable and reliable**

DEPLOYMENT

ACTIVITY 7

FOLDER NAME: activity-7-<LAST-NAME>
branch name: activity-7

Activity 7 - Testing with Vitest and Deploying Your App

This is a continuation of your full stack project from Activities 5 and 6.

Testing with Vitest

1. Install Vitest and testing libraries: ***npm install -D vitest @testing-library/react @testing-library/jest-dom***
2. Update your **vite.config.js** to enable testing.
3. Create a test file for all of your components (e.g., **ComponentName.test.jsx**)
4. Create at least 3 test cases for each file and make sure it will all passed.
5. Make sure all tests pass before deployment.

Activity 7 - Testing with Vitest and Deploying Your App

This is a continuation of your full stack project from Activities 4, 5, and 6.

Deployment

For frontend (React only):

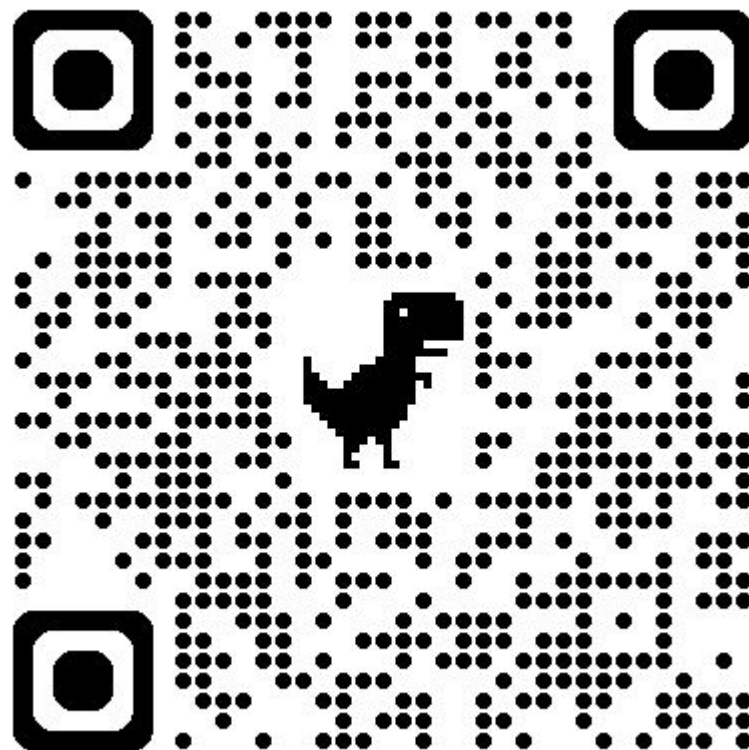
1. Push your frontend code to GitHub.
2. Deploy using [Vercel](#) or [Netlify](#) or [Render](#).
3. Copy your live URL.

Finalize and Submit

4. Add your deployed app URL to the **README.md** file.
5. Include concise documentation in your README:
 - App purpose or summary
 - Technologies used
 - Setup instructions
 - Deployment link
 - Author/team name
6. Pass your GitHub repository and live URL to **Ms. Anna** once done.

EVALUATION - REACTJS

MAY 5 - 8, 2026





Reach Us!

Visit Us

12th/F The Trade & Financial Tower Unit 1206
32nd Street & 7th Avenue Bonifacio Global City,
Taguig 1634 Philippines

Email Us

inquiry@trainosys.com

Browse Our Website

www.trainosys.com





TRAINOSYS

Training the Future Today

